

CampusGuard

COS 214 - Prac 5

Members:

Corné Myburgh - u25505204

Felicity Moyo - u24702791

Hayley Nel - u25101821

Task 1

1.1 Incident-response responsibilities

CampusGuard supports the coordination of emergency incidents across a large university campus. The system supports the following responsibilities:

Incident management: Create, track, and update incidents. Created Incidents are in the 'Started' state. Once they get reported every response team gets notified. After a team has been dispatched the Incident goes to 'Resolved' state.

Medical response: The Medical Team responds to medical-related emergencies and provides the required medical assistance.

Security and evacuation: The Security Team responds to security incidents, dispatches personnel, and coordinates campus evacuations when required.

FireDepartment: The Fire Department is an external service that is responsible for handling fire emergencies that may occur on Campus.

Campus control over infrastructure: The Campus Control Server can close or reopen affected buildings and campus areas and activate backup power during emergencies.

Communication: Response components communicate their actions during an incident to show what each team is doing.

Operations control: The Operations Dashboard allows authorised operators to issue actions such as dispatching response teams, requesting evacuations, and cancelling operations.

1.3 Design and GOF Participants

Command		
Invoker	OperationsDashboard	
Command	OperationAction	
ConcreteCommand	DispatchUnit, Evacuate, CancelAction	

Command		
Receiver	EmergencyChannel	EmergencyChannel is the ConcreteMediator that communicates the command to all its colleagues.

Command issues commands to the Mediator pattern. The mediator calls the corresponding functions for all the units.

Mediator		
Mediator	Radio	
ConcreteMediator	EmergencyChannel	
Colleague	Unit	
ConcreteColleague	MedicalTeam, SecurityTeam, CampusControlServer, Incident, CommunicationService	

The mediator acts as the communication medium between all the response units. Gets used by the Command to send actions to units.

Adapter		
Target	Unit	
Adapter	CommunicationService	
Adaptee	FireDepartment	

Adapts the external FireDepartment to integrate correctly with the CampusGuard system.

Facade		
Facade	IncidentHandler	
Subsystems	EmergencyChannel, CampusControlServer, ServerManager, OperationsDashboard	

Used by the Client to easily create and manage incidents within the campus, dispatch units from the OperationsDashboard or manage the CampusControlserver.

1.4 Additional Patterns

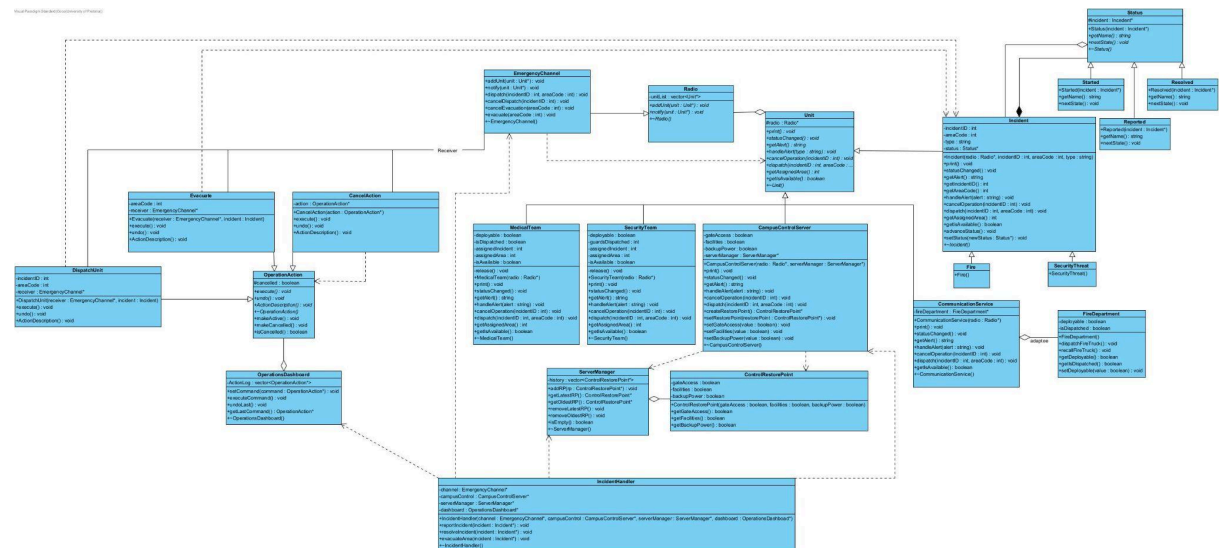
State		
Context	Incident	
State	Status	
ConcreteState	Started, Reported, Resolved	

The State pattern is used to show the state of the incident and control the transitions of it. When an Incident gets created it is in the 'Started' state. It advances from the Started to the Reported state. After a team gets dispatched it advances to the Resolved state. This clearly shows what is happening to the Incident and the flow within it.

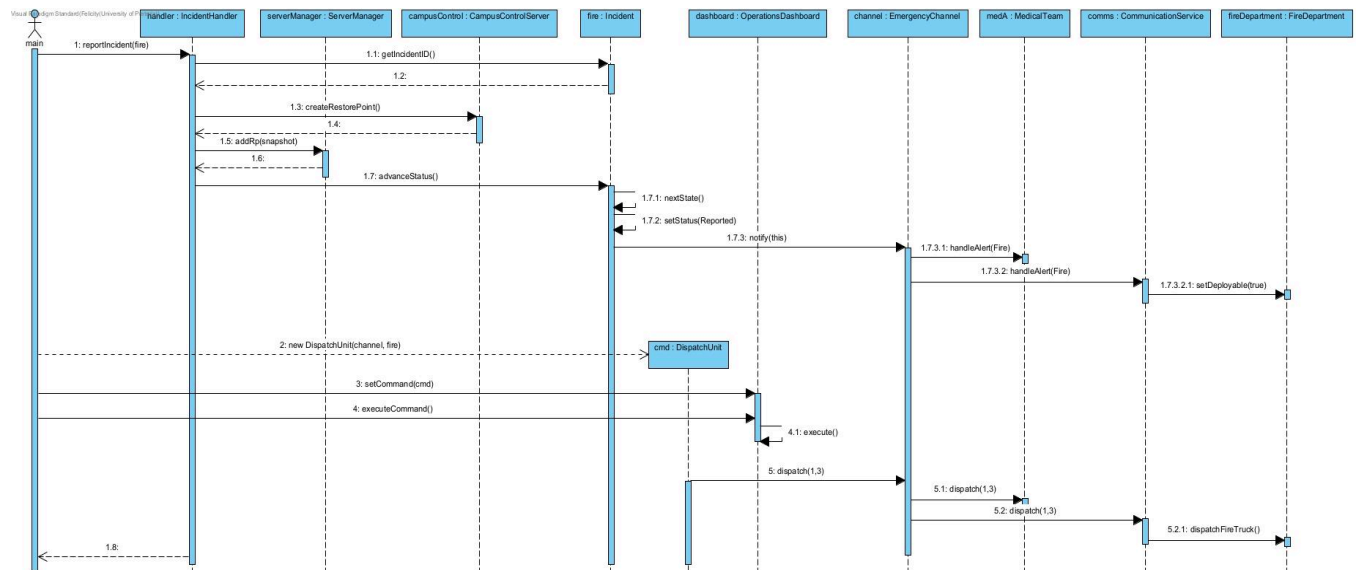
Memento		
Originator	CampusControlServer	
Memento	ControlRestorePoint	
Caretaker	ServerManager	

The Memento integrates perfectly with the CampusControlServer. The CampusControlServer has boolean values for gateAccess, facilities, backupPower. This means when an Incident occurs it will go into a safety protocol and change the original boolean values. The memento stores its state right before the incident and when the incident gets resolved it can use this to restore itself back to its previous state. The concept of memento also works well with the idea that it is a Server that needs to have a restorePoint to backup to.

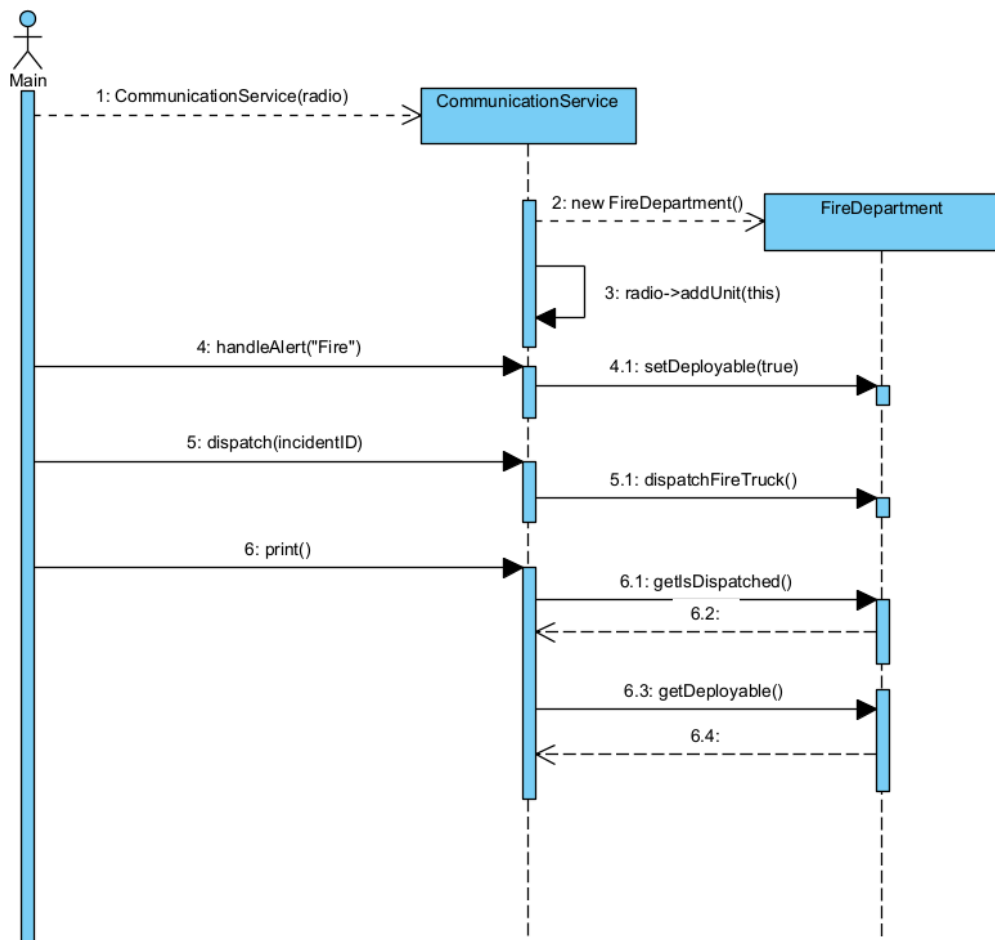
Mediator and Subsystems: The mediator directly causes changes in the patterns of its children. When a mediator sends an alert it causes Units like CampusControlServer to create the backup and later restore its point.



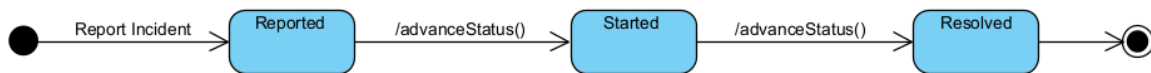
4.2 UML Sequence Diagram with Command



4.3 UML Sequence Diagram with Adapter



4.4 Behavioural Diagram (State diagram)



5.5 Valgrind Evidence

```
==54227== HEAP SUMMARY:
==54227==    in use at exit: 0 bytes in 0 blocks
==54227== total heap usage: 52 allocs, 52 frees, 75,894 bytes allocated
==54227==
==54227== All heap blocks were freed -- no leaks are possible
==54227==
==54227== For lists of detected and suppressed errors, rerun with: -s
==54227== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

5.6 GDB Evidence

```
g++ -g -std=c++11 -Wall -Wextra -c MedicalTeam.cpp -o MedicalTeam.o
In file included from MedicalTeam.h:4,
                 from MedicalTeam.cpp:1:
Unit.h: In member function 'virtual void Unit::cancelOperation(int)':
Unit.h:18:42: warning: unused parameter 'incidentID' [-Wunused-parameter]
   18 |     virtual void cancelOperation(int incidentID){
      |                                ~~~~~^~~~~~
Unit.h: At global scope:
Unit.h:18:22: warning: 'virtual void Unit::cancelOperation(int)' was hidden [-Woverloaded-virtual=]
   18 |     virtual void cancelOperation(int incidentID){
      |
MedicalTeam.h:23:10: note: by 'void MedicalTeam::cancelOperation()'
   23 |     void cancelOperation();
      |
```

Cancelling the false alarm

Unit: cancelOperation default

```
felicity@DESKTOP-G3PGM47:~/COS 214 Projects/CampusGuard-Prac5$ gdb ./CampusGuard
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./CampusGuard...
(gdb)
```

```
(gdb) break MedicalTeam::cancelOperation
Breakpoint 1 at 0x9f18: file MedicalTeam.cpp, line 47.
(gdb) run
```

```
(gdb) info vtbl this
vtable for 'MedicalTeam' @ 0x55555569888 (subobject @ 0x5555557d860):
[0]: 0x5555555db46 <MedicalTeam::print()>
[1]: 0x5555555dc16 <MedicalTeam::statusChanged()>
[2]: 0x5555555dc26 <MedicalTeam::getAlert[abi:cxx11]()>
[3]: 0x5555555dd06 <MedicalTeam::handleAlert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >>>
[4]: 0x5555555df40 <MedicalTeam::dispatch(int, int)>
[5]: 0x55555558964 <Unit::cancelOperation(int)>
[6]: 0x5555555dfde <MedicalTeam::getAssignedArea()>
[7]: 0x5555555dff4 <MedicalTeam::getIsAvailable()>
```

Bug: warning: ‘virtual void Unit::cancelOperation(int)’ was hidden

[-Woverloaded-virtual=]

```
18 |     virtual void cancelOperation(int incidentID){
    |           ^~~~~~
```

MedicalTeam.h:23:10: note: by ‘void MedicalTeam::cancelOperation()’

```
23 |     void cancelOperation() ;
    |     ^~~~~~
```

Cause: MedicalTeam inherits from Unit and is meant to override the default cancelOperation function. In this case MedicalTeam’s cancelOperation parameter did not match Unit’s cancelOperation parameter and the override keyword was not included in the function definition. This meant the code was still compiled despite the bug.

Result of bug: Within the first runtime scenario in main, MedicalTeam’s cancelOperation function is called and instead of running MedicalTeam’s implementation, the default cancelOperation method from Unit was run instead. This is visible from MedicalTeam’s vtable.

Fix: made cancelOperation a pure virtual function with no default implementation, fixed MedicalTeam’s cancelOperation parameters to match the base class and added the override keyword to the function definition.

8.10 Github and Contribution

Github link:

<https://github.com/joyincity/CampusGuard-Prac5.git>

We had a meeting on how exactly we want every pattern to work together and got everyone on the same page, then divided some of the patterns among us. Each of us took one of the big diagrams to draw. We used github to combine all our work and pull requests to approve others’ work. We discussed integration issues or new ideas as a group to make decisions.

8.11 Contribution Statements

Corné Myburgh - UML Class Diagram, Implemented Memento and Mediator, Docker

Felicity Moyo - Command Sequence Diagram, Implemented Command and Facade, GDB

Hayley Nel - Adapter Sequence Diagram, State Diagram, Implemented State and Adapter

Team Effort:

PDF document and Design Ideas

Docker

Running normally

Build the application:

```
make
```

Run the application:

```
make run
```

Clean the compiled files:

```
make clean
```

Running with Docker

Build and run the application using Docker Compose:

```
docker compose up --build
```

To stop the application:

```
docker compose down
```

Docker Build

To build the Docker image manually:

```
docker build -t campusguard .
```

To run the Docker image:

```
docker run --rm campusguard
```